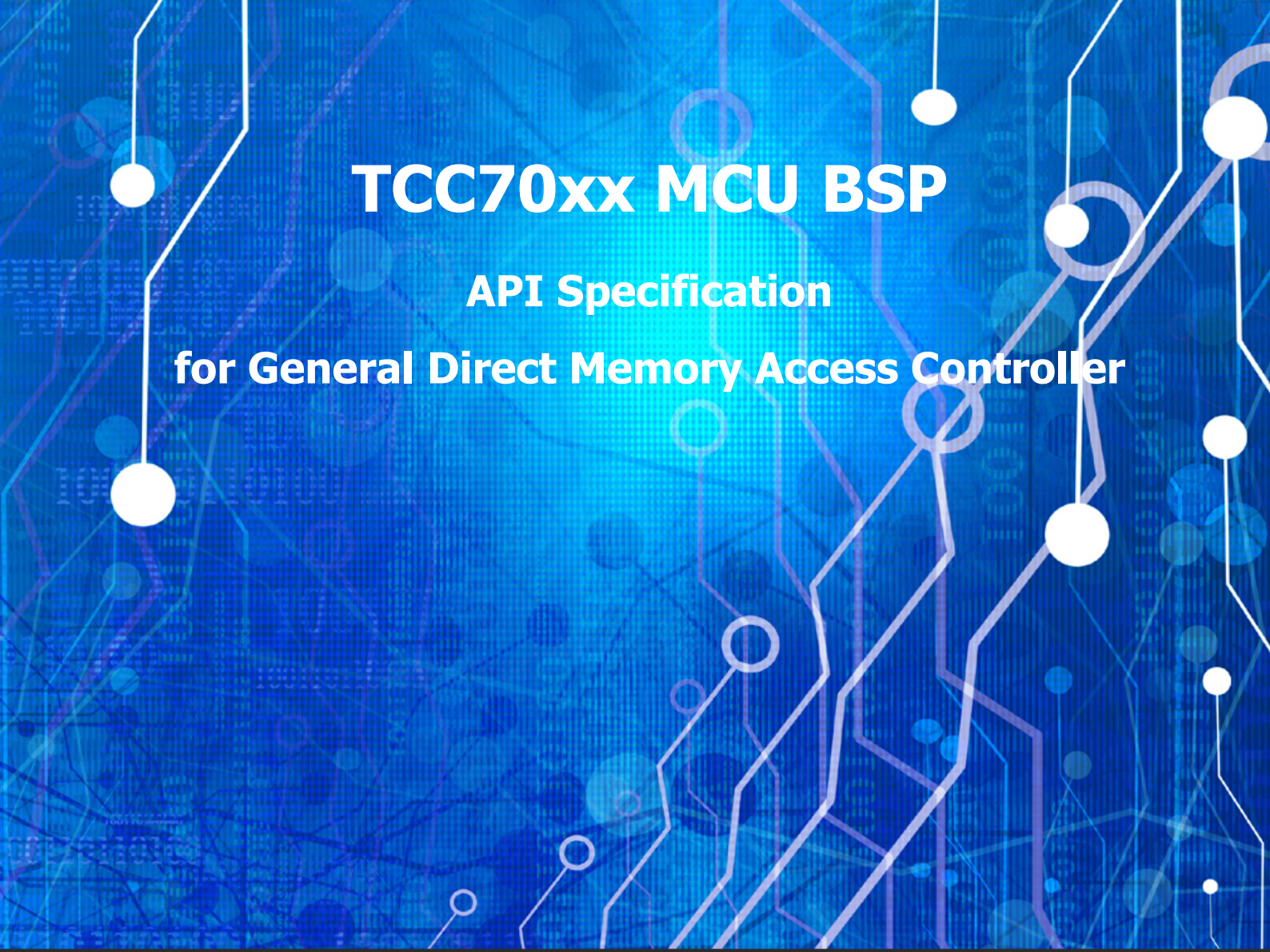




TCC70xx MCU BSP

API Specification

for General Direct Memory Access Controller

Rev. 0.10 [A]

2021-09-03

Preliminary version

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit www.telechips.com for more information.

© 2021 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Source Files.....	4
3.1	Location of Source Files.....	4
3.2	Simple Description of Source Files.....	4
4	Constants.....	5
4.1	GDMA_CON_MAX	5
4.2	GDMA_CH_MAX.....	5
4.3	GDMA Peripheral ID.....	6
5	Structures	7
5.1	GDMAInformatoin_t.....	7
6	APIs	8
6.1	GDMA_Init.....	8
6.2	GDMA_TransferMem2Mem.....	9
6.3	GDMA_InterruptEnable	10
6.4	GDMA_InterruptDisable.....	11
6.5	GDMA_SetAddrIncrement.....	12
6.6	GDMA_SetTransferWidth	13
6.7	GDMA_SetBurstSize	14
6.8	GDMA_SetTransferSize	15
6.9	GDMA_SetFlowControl	16
6.10	GDMA_Reset	17
6.11	GDMA_SetPeri.....	18
6.12	GDMA_GetTransferSize.....	19
6.13	GDMA_ChannelEnable.....	20
6.14	GDMA_ChannelDisable	21
6.15	GDMA_SetSrcAddr	22
6.16	GDMA_SetDestAddr	23
6.17	GDMA_SetIliAddr	24
7	How to use DMA Driver	25
7.1	Operation of Memory to Memory (M2M) DMA Transfer	25
7.2	Initialize GDMA Driver	25
7.3	Request M2M DMA Transfer.....	26
8	References	27
9	Revision History	28
	Rev. 0.10: 2021-09-03	28

Figures

Figure 3.1 Location of Source Files	4
Figure 7.1 Activity Diagram for GDMA Driver	25

Tables

Table 2.1 Terms and Abbreviation	3
--	---

1 INTRODUCTION

This document describes the general direct memory access controller device driver (hereinafter referred to as GDMA driver) of TCC70xx MCU BSP. For more detailed information on direct memory access controller in the TCC70xx MCU, refer to Chapter 14 General Direct Memory Access (DMA) Controller in "*TCC70xx Full Specification*". [1]

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

DMA	Direct Memory Access
GDMA	General Direct Memory Access (DMA) Controller
ISR	Interrupt Service Routine
M2M	Memory to Memory

3 SOURCE FILES

3.1 Location of Source Files

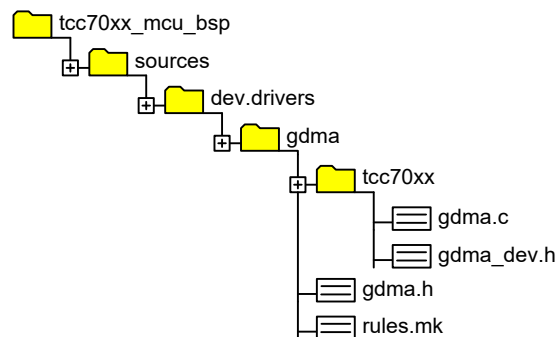


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- `gdma.c`
 - GDMA driver source
- `gdma_dev.h`
 - GDMA hardware definition header
- `gdma.h`
 - GDMA driver API header
- `rules.mk`
 - Rules used for build

4 CONSTANTS

4.1 GDMA_CON_MAX

Constant value for maximum GDMA controllers

Declaration

```
#define GDMA_CON_MAX (8UL)
```

Description

Value	Description
GDMA_CON_MAX	<i>The maximum number of controllers. There are eight General DMA controllers in TCC70xx.</i>

Remark

ARM Primecell PL081 is the IP used by GDMA and refer to the document below.

- <http://infocenter.arm.com/help/topic/com.arm.doc.ddi0218e/DDI0218.pdf>

4.2 GDMA_CH_MAX

Constant value for maximum GDMA channels

Declaration

```
#define GDMA_CH_MAX (2UL)
```

Description

Value	Description
GDMA_CH_MAX	<i>The maximum number of channels. Each DAM controller has two DMA channels.</i>

Remark

None

4.3 GDMA Peripheral ID

Peripheral ID that can be requested of DMA

Declaration

```
#define GDMA_PERI_REQ_PORT_UART0_RX (0UL)
#define GDMA_PERI_REQ_PORT_UART0_TX (1UL)

#define GDMA_PERI_REQ_PORT_UART1_RX (0UL)
#define GDMA_PERI_REQ_PORT_UART1_TX (1UL)

#define GDMA_PERI_REQ_PORT_UART2_RX (0UL)
#define GDMA_PERI_REQ_PORT_UART2_TX (1UL)

#define GDMA_PERI_REQ_PORT_UART3_RX (0UL)
#define GDMA_PERI_REQ_PORT_UART3_TX (1UL)

#define GDMA_PERI_REQ_PORT_UART4_RX (0UL)
#define GDMA_PERI_REQ_PORT_UART4_TX (1UL)

#define GDMA_PERI_REQ_PORT_UART5_RX (0UL)
#define GDMA_PERI_REQ_PORT_UART5_TX (1UL)

#define GDMA_PERI_REQ_PORT_ICTC0 (2UL)
#define GDMA_PERI_REQ_PORT_ICTC1 (3UL)
#define GDMA_PERI_REQ_PORT_ICTC2 (4UL)
#define GDMA_PERI_REQ_PORT_ICTC3 (5UL)
#define GDMA_PERI_REQ_PORT_ICTC4 (6UL)
#define GDMA_PERI_REQ_PORT_ICTC5 (7UL)

#define GDMA_PERI_REQ_PORT_ADC0 (8UL)
#define GDMA_PERI_REQ_PORT_ADC1 (9UL)

#define GDMA_PERI_REQ_PORT_CAN0 (10UL)
#define GDMA_PERI_REQ_PORT_CAN1 (11UL)
#define GDMA_PERI_REQ_PORT_CAN2 (12UL)
```

Description

Value	Description
GDMA_PERI_REQ_PORT_nnn	Peripheral ID that can be requested of DMA. Related DAM request signal is described in Table 14.2 of "TCC70xx Full Specification".

Remark

None

5 STRUCTURES

5.1 GDMAInformatoin_t

Structure for controlling GDMA driver. Each variable must be properly initialized and must be maintained during run time.

Declaration

```
typedef struct GDMA_INFORMATION
{
    uint32    iCon;           // controller id (0 ~ GDMA_CON_MAX).
    uint32    iCh;           // channel id (0 ~ GDMA_CH_MAX)
    uint8 *   iSrcAddr;      // sourced address.
    uint8 *   iDestAddr;     // destination address.
    uint32    iBufSize;      // the length of the buffer for DMA transferring.
    uint32    iTransSize;    // transfer request length
} GDMAInformation_t;
```

Description

Value	Description
iCon	This value indicates the controller number for transferring the DMA request. Value is valid only in the range 0 to GDMA_CON_MAX.
iCh	This value indicates the channel number for transferring the DMA request. Value is valid only in the range 0 to GDMA_CH_MAX.
iSrcAddr	This value indicates the source address.
iDestAddr	This value indicates the destination address.
iBufSize	This value indicates the size of the DMA buffer in byte.
iTransSize	This value indicates the size of requested bytes to be transferred.

Remark

None

6 APIs

6.1 GDMA_Init

Initializes the GDMA. There are two parameters in this function.

- The first parameter is for controlling the driver, and you should specify the controller and channel index for this transfer using the "iCon" and "iCh" member variables in the first parameter.
- The second parameter is for changing the priority of Interrupt Service Routine (ISR). If you want to use the default value, use **GIC_PRIORITY_NO_MEAN** as input value.

Declaration

```
sint32 GDMA_Init  
(  
    GDMAInformation_t *    sDmacon,  
    uint32                 uiIntPrio  
);
```

Parameters

Value	Description
sDmacon	[in] Pointer of GDMA control data. Refer to GDMAInformation_t .
uiIntPrio	[in] Priority of GDMA interrupt

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

6.2 GDMA_TransferMem2Mem

Copies data of source memory to destination memory. The first parameter (sDmacon) is a pointer that points the control data, and the control data is passed variable when the driver is initialized.

Declaration

```
sint32 GDMA_TransferFunctionForM2M  
(  
    GDMAInformation_t *    sDmacon,  
    uint32                 uiSrcAddr,  
    uint32                 uiDestAddr,  
    uint32                 uiLen  
);
```

Parameters

Value	Description
sDmacon	[in] Pointer of GDMA control data. Refer to GDMAInformation t.
uiSrcAddr	[in] Source address. This address is valid only for physical address.
uiDestAddr	[in] Destination address. This address is valid only for physical address.
uiLen	[in] Length of data requested for transfer

Return

Value	Description
SAL_RET_SUCCESS	Success
SAL_RET_FAILED	Failure

Remark

None

6.3 GDMA_InterruptEnable

Enables the GDMA interrupt.

Declaration

```
void GDMA_InterruptEnable
(
    GDMAInformation_t *  sDmacon
);
```

Parameters

Value	Description
sDmacon	[in] Pointer of GDMA control data. Refer to GDMAInformation t .

Return

None

Remark

None

6.4 GDMA_InterruptDisable

Disables the GDMA interrupt.

Declaration

```
void GDMA_InterruptDisable
(
    GDMAInformation_t *   sDmacon
);
```

Parameters

Value	Description
<i>sDmacon</i>	<i>[in]</i> Pointer of GDMA control data. Refer to GDMAInformation t .

Return

None

Remark

None

6.5 GDMA_SetAddrIncrement

Sets the increment attribute of source and destination.

Declaration

```
void GDMA_SetAddrIncrement
(
    GDMAInformation_t *   sDmacon,
    uint8                 ucDestInc,
    uint32                 uiSrcInc
);
```

Parameters

Value	Description
sDmacon	[in] Pointer of GDMA control data. Refer to GDMAInformation_t .
ucDestInc	[in] Destination increment. When setting GDMA_INC, destination address is incremented after each transfer. Possible parameters are as follows. #define GDMA_INC (1U) #define GDMA_NO_INC (0U)
uiSrcInc	[in] Source increment. When setting GDMA_INC, source address is incremented after each transfer. Possible parameters are as follows. #define GDMA_INC (1U) #define GDMA_NO_INC (0U)

Return

None

Remark

None

6.6 GDMA_SetTransferWidth

Sets the transfer width of data bus for source and destination.

Declaration

```
void GDMA_SetTransferWidth
(
    GDMAInformation_t *   sDmacon,
    uint8                 ucDestWidth,
    uint32                 uiSrcWidth
);
```

Parameters

Value	Description
sDmacon	[in] Pointer of GDMA control data. Refer to GDMAInformation_t .
ucDestWidth	[in] Destination transfer width. Possible parameters are as follows: #define GDMA_TRANSFER_SIZE_BYTE (0U) #define GDMA_TRANSFER_SIZE_HALF (1U) #define GDMA_TRANSFER_SIZE_WORD (2U)
uiSrcWidth	[in] Source transfer width. Possible parameters are as follows: #define GDMA_TRANSFER_SIZE_BYTE (0U) #define GDMA_TRANSFER_SIZE_HALF (1U) #define GDMA_TRANSFER_SIZE_WORD (2U)

Return

None

Remark

None

6.7 GDMA_SetBurstSize

Sets the burst transfer width of source and destination.

Declaration

```
void GDMA_SetBurstSize
(
    GDMAInformation_t *   sDmacon,
    uint8                 ucDestBurst,
    uint32                 uiSrcBurst
);
```

Parameters

Value	Description
<i>sDmacon</i>	[in] Pointer of GDMA control data. Refer to GDMAInformation_t .
<i>ucDestBurst</i>	[in] Destination burst size. Possible parameters are as follows: <pre>#define GDMA_BURST_SIZE_1 (0U) #define GDMA_BURST_SIZE_4 (1U) #define GDMA_BURST_SIZE_8 (2U) #define GDMA_BURST_SIZE_16 (3U) #define GDMA_BURST_SIZE_32 (4U) #define GDMA_BURST_SIZE_64 (5U) #define GDMA_BURST_SIZE_128 (6U) #define GDMA_BURST_SIZE_256 (7U)</pre>
<i>uiSrcBurst</i>	[in] Source burst size. Possible parameters are as follows: <pre>#define GDMA_BURST_SIZE_1 (0U) #define GDMA_BURST_SIZE_4 (1U) #define GDMA_BURST_SIZE_8 (2U) #define GDMA_BURST_SIZE_16 (3U) #define GDMA_BURST_SIZE_32 (4U) #define GDMA_BURST_SIZE_64 (5U) #define GDMA_BURST_SIZE_128 (6U) #define GDMA_BURST_SIZE_256 (7U)</pre>

Return

None

Remark

None

6.8 GDMA_SetTransferSize

Sets the transfer size.

Declaration

```
void GDMA_SetTransferSize
(
    GDMAInformation_t *    sDmacon,
    uint32                 uiTransferSize
);
```

Parameters

Value	Description
sDmacon	[in] Pointer of GDMA control data. Refer to GDMAInformation t .
uiTransferSize	[in] Transfer size. Value is valid only in the range 1 to 0xFFF.

Return

None

Remark

None

6.9 GDMA_SetFlowControl

Sets the flow control type of channel configuration register.

Declaration

```
void GDMA_SetFlowControl
(
    GDMAInformation_t *    sDmacon,
    uint32                 uiFlow
);
```

Parameters

Value	Description
sDmacon	[in] Pointer of GDMA control data. Refer to GDMAInformation t .
uiFlow	[in] Flow control and transfer type. Possible parameters are as follows: <pre>#define GDMA_FLOW_TYPE_M2M (0UL) #define GDMA_FLOW_TYPE_M2P (1UL) #define GDMA_FLOW_TYPE_P2M (2UL) #define GDMA_FLOW_TYPE_P2P (3UL) #define GDMA_FLOW_TYPE_P2P_BY_DP (4UL) #define GDMA_FLOW_TYPE_M2P_BY_P (5UL) #define GDMA_FLOW_TYPE_P2M_BY_P (6UL) #define GDMA_FLOW_TYPE_P2P_BY_SP (7UL)</pre> Flow controller can be: ■ DMA ■ Source Peripheral (SP) ■ Destination Peripheral (DP) Transfer type can be: ■ Memory to Memory (M2M) ■ Memory-to-Peripheral (M2P) ■ Peripheral-to-Memory (P2M) ■ Peripheral-to-Peripheral (P2P)

Return

None

Remark

None

6.10 GDMA_Reset

Resets the GDMA controller given by parameter.

Declaration

```
SALRetCode_t GDMA_Reset  
(  
    uint8 ucCon  
);
```

Parameters

Value	Description
<i>ucCon</i>	<i>[in] Controller ID to be reset</i>

Return

Value	Description
<i>SAL_RET_SUCCESS</i>	<i>Success</i>
<i>SAL_RET_FAILED</i>	<i>Failure</i>

Remark

None

6.11 GDMA_SetPeri

Sets the peripheral type value. The possible peripheral type is defined in Chapter 4.3.

Declaration

```
void GDMA_SetPeri
(
    GDMAInformation_t *   sDmacon,
    uint8                 ucDestPeri,
    uint8                 ucSrcPeri
);
```

Parameters

Value	Description
<i>sDmacon</i>	<i>[in]</i> Pointer of GDMA control data. Refer to GDMAInformation_t .
<i>ucDestPeri</i>	<i>[in]</i> Destination peripheral. Possible parameters are defined in Chapter 4.3
<i>ucSrcPeri</i>	<i>[in]</i> Source peripheral. Possible parameters are defined in Chapter 4.3

Return

None

Remark

None

6.12 GDMA_GetTransferSize

Gets the current transfer length of GDMA.

Declaration

```
uint32 GDMA_GetTransferSize
(
    GDMAInformation_t * sDmacon
);
```

Parameters

Value	Description
<i>sDmacon</i>	<i>[in]</i> Pointer of GDMA control data. Refer to GDMAInformation_t .

Return

Value	Description
<i>uint32</i>	Return the current transfer length. It means the remained data length that to be transferred. The actual transferred length is calculated by “request transfer length - GDMA_GetTransferSize()” formula.

Remark

None

6.13 GDMA_ChannelEnable

Enables the channel.

Declaration

```
void GDMA_ChannelEnable
(
    GDMAInformation_t *  sDmacon
);
```

Parameters

Value	Description
sDmacon	[in] Pointer of GDMA control data. Refer to GDMAInformation t .

Return

None

Remark

None

6.14 GDMA_ChannelDisable

Disables the channel.

Declaration

```
void GDMA_ChannelDisable
(
    GDMAInformation_t *  sDmacon
);
```

Parameters

Value	Description
sDmacon	[in] Pointer of GDMA control data. Refer to GDMAInformation t .

Return

None

Remark

None

6.15 GDMA_SetSrcAddr

Sets the source address.

Declaration

```
void GDMA_SetSrcAddr
(
    GDMAInformation_t *  sDmacon,
    uint32               uiAddr
);
```

Parameters

Value	Description
<i>sDmacon</i>	<i>[in]</i> Pointer of GDMA control data. Refer to GDMAInformation_t .
<i>uiAddr</i>	<i>[in]</i> Source address. This address is valid only for physical address (non-cacheable).

Return

None

Remark

None

6.16 GDMA_SetDestAddr

Sets the destination address.

Declaration

```
void GDMA_SetDestAddr
(
    GDMAInformation_t *  sDmacon,
    uint32               uiAddr
);
```

Parameters

Value	Description
sDmacon	[in] Pointer of GDMA control data. Refer to GDMAInformation t .
uiAddr	[in] Destination address. This address is valid only for physical address (non-cacheable).

Return

None

Remark

None

6.17 GDMA_SetLLiAddr

Sets the linked list item on the Linked List Item Register. The linked list item data structures must be four-word aligned.

Declaration

```
void GDMA_SetLLiAddr
(
    GDMAInformation_t *    sDmacon,
    uint32                 uiLLIAddr
);
```

Parameters

Value	Description
sDmacon	[in] Pointer of GDMA control data. Refer to GDMAInformation t .
uiLLIAddr	[in] Address of Linked List item header

Return

None

Remark

None

7 HOW TO USE DMA DRIVER

To use the GDMA driver, the ***GDMA_Initialize()*** function must be called first. For initialization of GDMA driver, you should prepare the variable of ***GDMAControlData_t*** type and initialize the channel parameter (***cdChannel***).

The ***GDMA_TransferFunctionForM2M()*** function provides the Memory to Memory (M2M) DMA transfer with blocking mode. The meaning of blocking mode is that the ***GDMA_TransferFunctionForM2M()*** function will not be returned until the transfer is completed. The ***GDMA_TransferFunctionForM2M()*** function introduces the OS-event-flag scheme for this blocking mode. Therefore, the user task that calls this API is pending in the suspend mode.

7.1 Operation of Memory to Memory (M2M) DMA Transfer

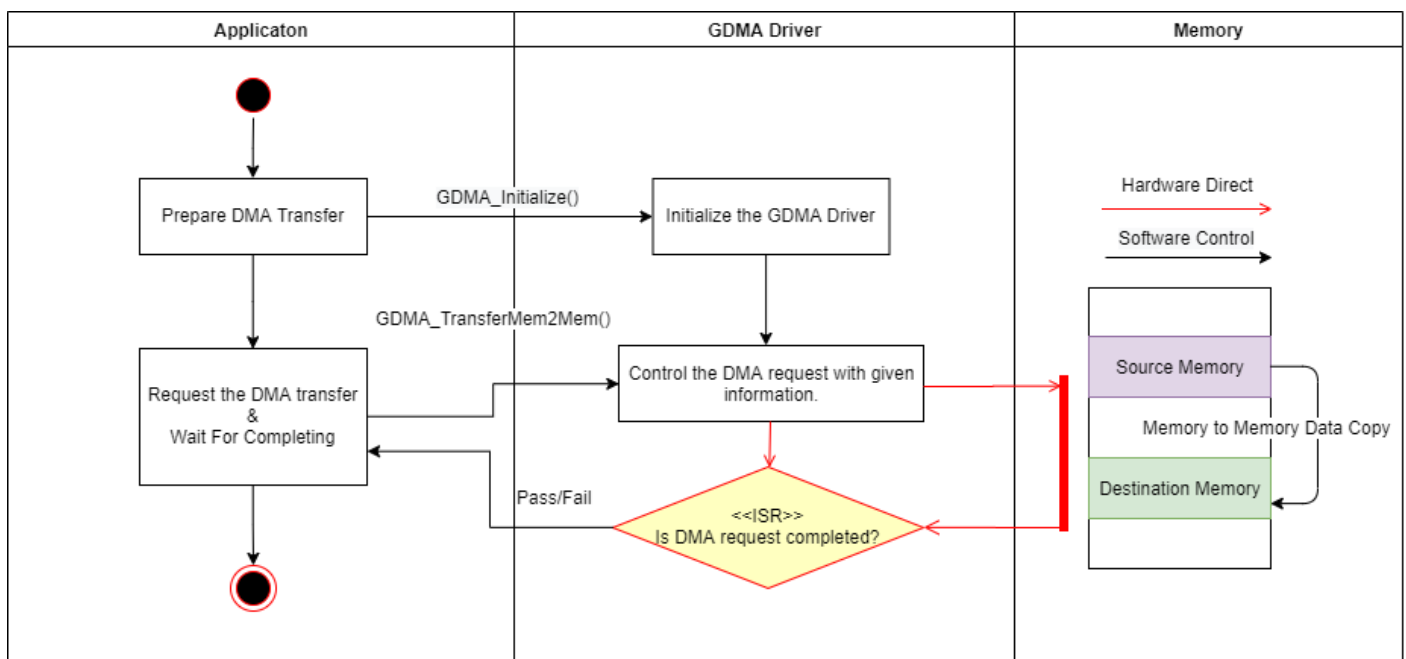


Figure 7.1 Activity Diagram for GDMA Driver

7.2 Initialize GDMA Driver

```

{
    static GDMAInformation_t sample_pGdmaControl;;
    sint32      ret;

    sample_pGdmaControl.iCon = 0;
    sample_pGdmaControl.iCh= 0;
    ret= GDMA_Init(&sample_pGdmaControl, GIC_PRIORITY_NO_MEAN);
}
  
```

7.3 Request M2M DMA Transfer

```
{  
    GDMAInformation_t sample_pGdmaControl;  
    uint32      request_size;  
    uint32      src_base;  
    uint32      dest_base;  
  
    request_size = (1024 + (4*16) + 3);  
    src_base     = 0xC1000000; //non-cacheable memory.  
    dest_base    = (0xC1000000 + 0x1000); //non-cacheable memory.  
    sample_pGdmaControl.iCon = 0;  
    sample_pGdmaControl.iCh = 0;  
  
    ret = GDMA_TransferFunctionForM2M(&sample_pGdmaControl, src_base, dest_base, request_size);  
}
```

8 REFERENCES

- [1] TCC70xx Full Specification
- [2] Contact Telechips for more details: sales@telechips.com

9 REVISION HISTORY

Rev. 0.10: 2021-09-03

- Preliminary version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."
"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."